

TP02 - B2 : Le diagramme de classe

1) Problématique	1
2) Exercices	1
3) TD4	8
4) TD5	10
5) Conclusion	12

1) Problématique

On cherche à comprendre le fonctionnement d'un diagramme de classe, pour cela nous allons en créer en utilisant le logiciel PowerAMC et en représentant certaines classes de façon graphique comme la classe LIVRE.

2) Exercices

Représentation de classe livre avec le diagramme de classe.

Nous devons représenter la classe livre suivante avec PowerAMC :

```
public class Livre {  
    private String ISBN,titre,Auteur;  
    private int prix;  
    public String getTitre() {..  
    public void setTitre(String titre) {..  
    public String getAuteur() {..  
    public void setAuteur(String auteur) {..  
    public int getPrix() {..  
    public void setPrix(int prix) {..  
  
    public Livre(String ISBN, String titre, String auteur, int prix) {..  
    public void Afficher() {..  
    public String getISBN() {..  
    public void setISBN(String ISBN) {..  
  
}
```

Nouveau

Type de modèle :

- Modèle Conceptuel de Données
- Modèle Libre
- Modèle Orienté Objet
- Modèle Physique de Données
- Modèle XML
- Modèle de Fluidité de l'Information
- Modèle de Gestion des Exigences
- Modèle de Processus Métiers
- Modèle de Traitements Merise

☒ Nouveau modèle ☐ Nouveau modèle depuis un template

Général Définitions étendues de modèle

Langage objet : Java

☒ Partager le langage objet ☐ Copier le langage objet dans le modèle

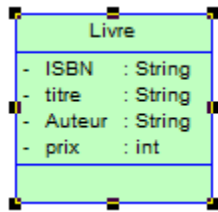
Premier diagramme : Diagramme de classes

OK Annuler Aide

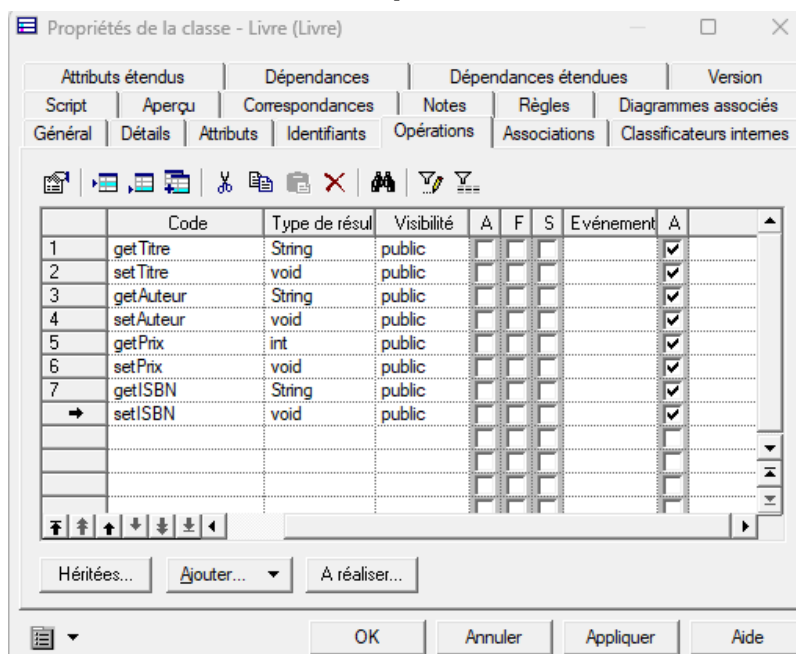
Livre

[illegible]

En comprenant cela, on peut déjà commencer à ajouter les attributs ISBN, titre, auteur et prix :

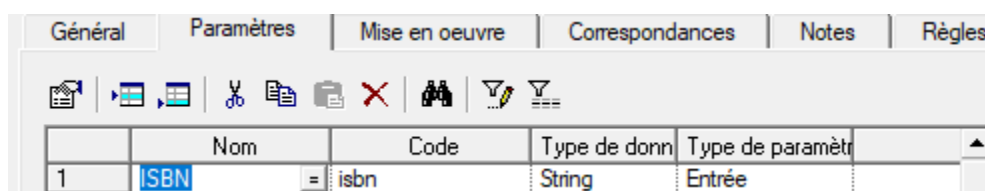


Ensuite il ne reste plus qu'à ajouter les méthodes (getters et setters) en allant dans la section **Opérations** :

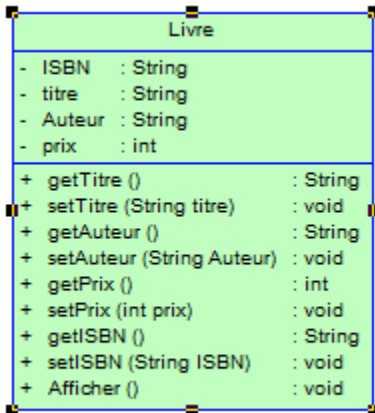


Attention, il ne faut pas oublier d'ajouter les paramètres des setters en sélectionnant le setter et en cliquant sur **Propriétés** .

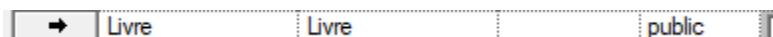
Lorsque c'est fait, il faut aller dans **Paramètres** et ajouter le paramètre correspondant au setter (ici pour setISBN par exemple) :



A partir de là ça devrait ressembler à ça :



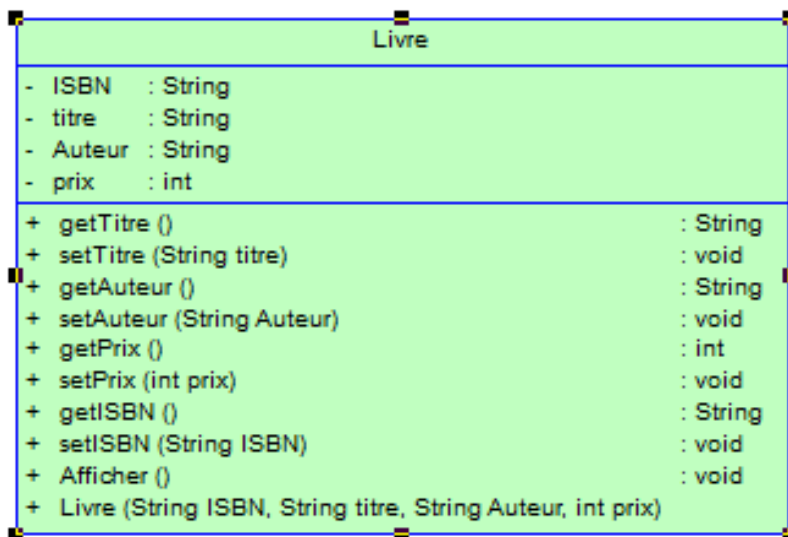
Il ne reste plus qu'à ajouter le constructeur dans l'onglet **Opérations**, il faut d'abord laisser le type de l'opération vide comme ceci :



Ensuite on va dans **Propriétés > Paramètres** puis on ajoute l'ISBN, le titre, l'auteur et le prix :

	Nom	Code	Type de donn	Type de paramètr
1	ISBN	ISBN	String	Entrée
2	titre	titre	String	Entrée
3	Auteur	Auteur	String	Entrée
→	<u>prix</u>	= prix	int	Entrée

Le résultat final doit ressembler à ça :



1. Ajout de classe Adh rent et association

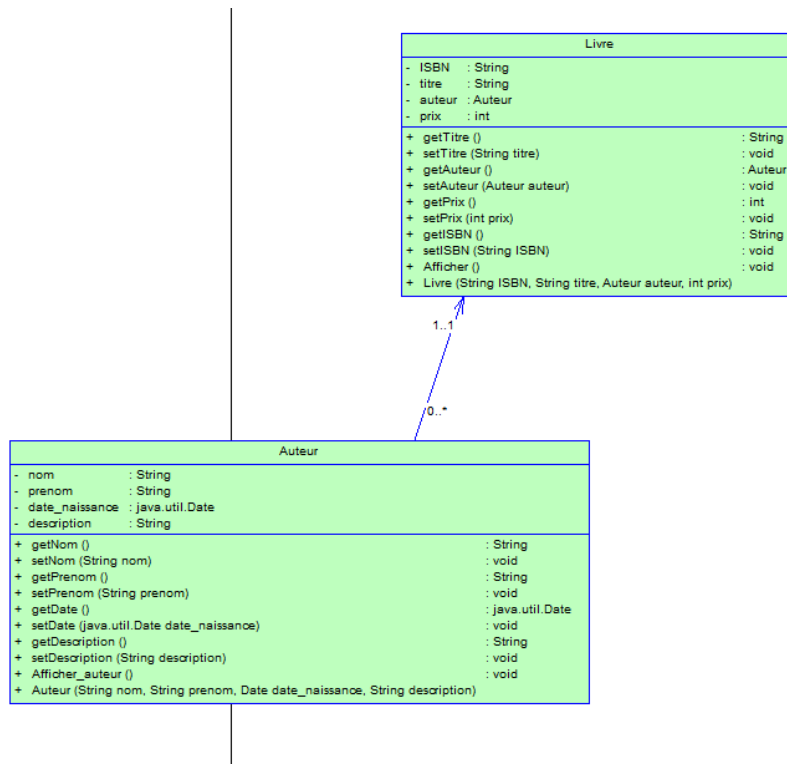
Maintenant nous souhaitons ajouter une classe Adh rent pour faire l'association avec la classe Livre. Cependant nous allons tout d'abord faire la deuxi me requ te qui demande de pouvoir avoir plus d'information sur un auteur, nous allons donc cr er une classe Auteur avec un constructeur, des getters et des setters :

Auteur		
- nom	: String	
- prenom	: String	
- date_naissance	: java.util.Date	
- description	: String	
+ getNom ()		: String
+ setNom (String nom)		: void
+ getPrenom ()		: String
+ setPrenom (String prenom)		: void
+ getDate ()		: java.util.Date
+ setDate (java.util.Date date_naissance)		: void
+ getDescription ()		: String
+ setDescription (String description)		: void
+ Afficher_auteur ()		: void
+ Auteur (String nom, String prenom, Date date_naissance, String description)		

Maintenant que c'est fait, nous allons modifier la classe Livre pour modifier l'attribut Auteur pour qu'il soit de classe Auteur (et il faut donc aussi modifier tout ce qui touche   l'attribut Auteur (qui est maintenant auteur)) :

Livre		
- ISBN	: String	
- titre	: String	
- auteur	: Auteur	
- prix	: int	
+ getTitre ()		: String
+ setTitre (String titre)		: void
+ getAuteur ()		: Auteur
+ setAuteur (Auteur auteur)		: void
+ getPrix ()		: int
+ setPrix (int prix)		: void
+ getISBN ()		: String
+ setISBN (String ISBN)		: void
+ Afficher ()		: void
+ Livre (String ISBN, String titre, Auteur auteur, int prix)		

Maintenant nous allons faire l'association entre Auteur et Livre, pour cela nous allons utiliser l'outil Association  et on relie les deux :

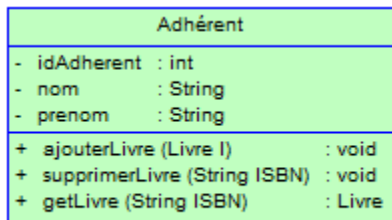


Ensuite lorsqu'on double clique sur la flèche on peut modifier les Multiplicités (cardinalités) dans la section **Détails** :

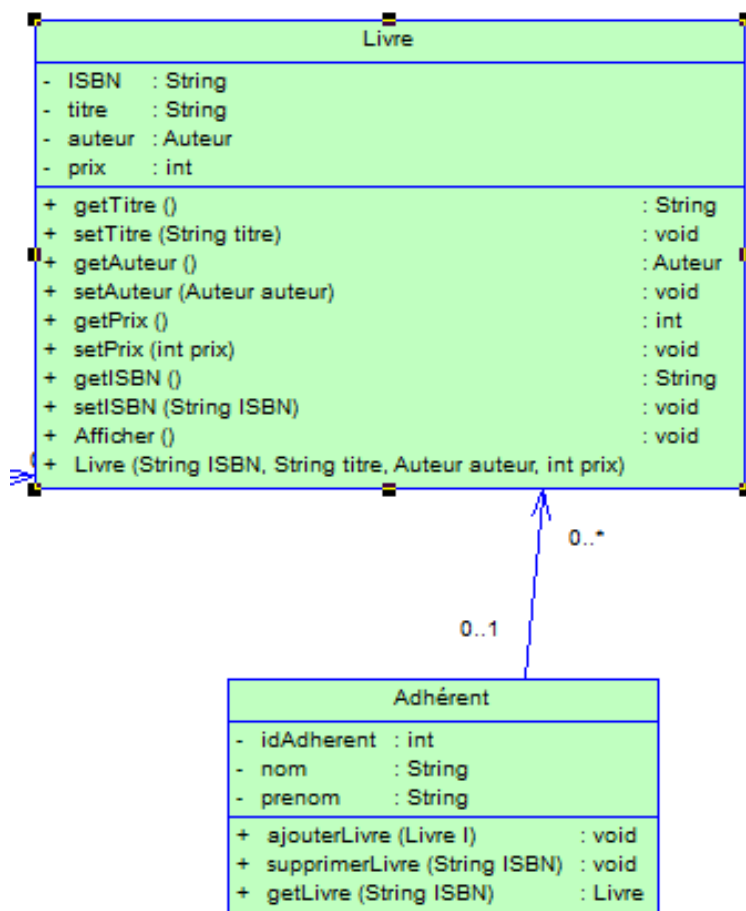
Auteur	Livre
Nom de rôle :	Nom de rôle :
Visibilité : public	Visibilité : public
Multiplicité : 0..*	Multiplicité : 1..1
Taille de tableau :	Taille de tableau :
Modifiable : Modifiable	Modifiable : Modifiable
Ordre : Non ordonné	Ordre : Non ordonné
Valeur initiale :	Valeur initiale :
☐ Navigable ☐ Persistant ☐ Volatile	☒ Navigable ☐ Persistant ☐ Volatile
Type de conteneur : <None>	Type de conteneur : <None>
Classe de mise en oeuvre : <None>	Classe de mise en oeuvre : <None>
Attribut migré : <Aucun>	Attribut migré : <Aucun>

Maintenant il faut choisir les bonnes cardinalités. Un auteur peut publier entre zéro et plusieurs livres. Un livre ne peut avoir qu'un auteur. Donc les cardinalités sont de 0, N (0 .. *) de Auteur à Livre et de 1,1 (1..1) de Livre à Auteur.

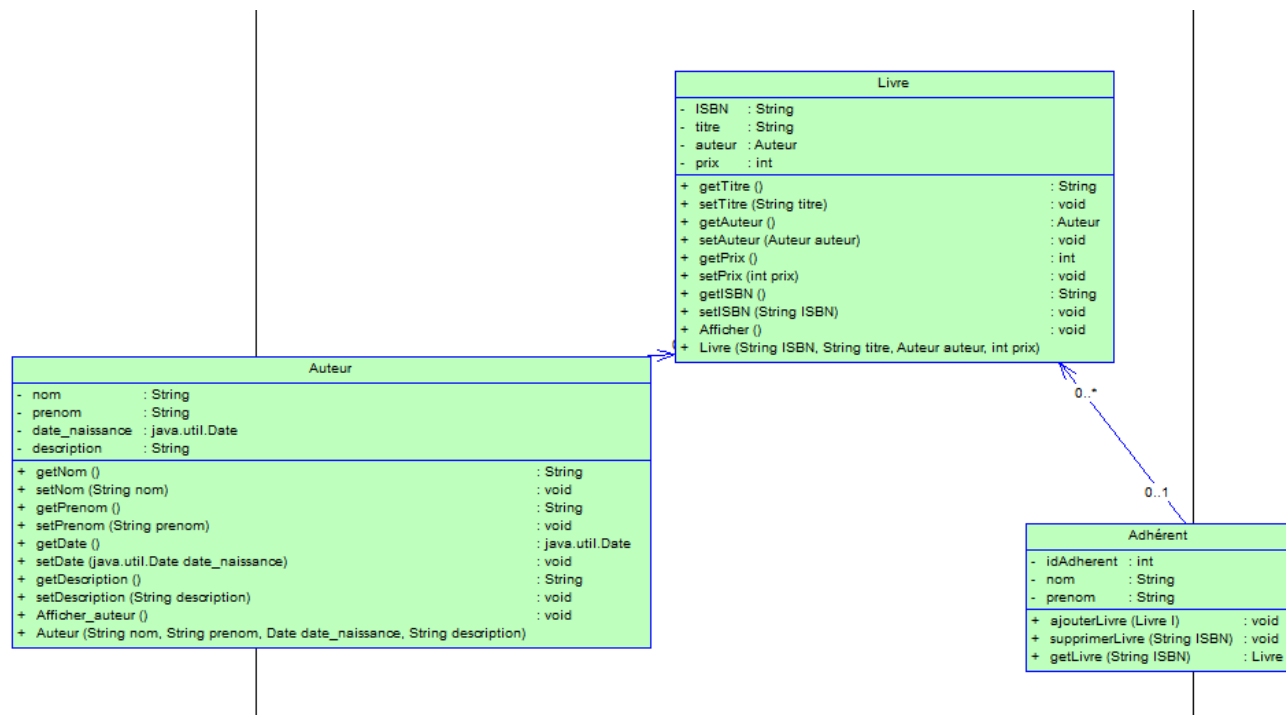
Maintenant que nous avons réussi à créer la classe Auteur et à faire l'association, il ne reste plus qu'à créer la classe Adhèrent :



Maintenant que la classe est créée il ne reste plus que l'association à faire (en sachant qu'un adhérent peut avoir plusieurs livres dans son dictionnaire (0,N) et qu'un livre peut être possédé par 0 ou 1 adhérent (0,1) :

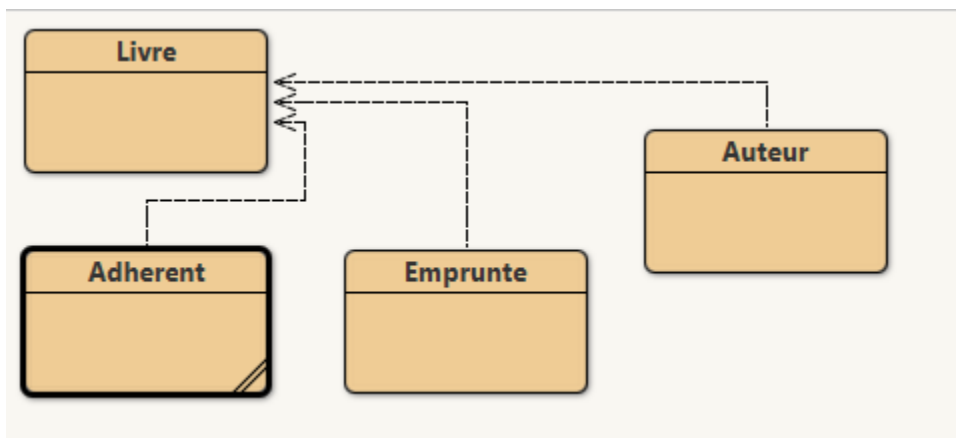


Le résultat final de la question 2 devrait être ceci :

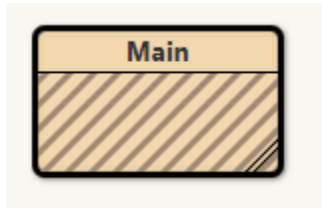


3) TD4

Pour ce TD nous allons utiliser le même diagramme que le précédent ce qui veut dire que le code aussi est le même, nous allons simplement faire en sorte de lancer le programme cette fois ci en incluant les .java générés avec PowerAMC et les inclure dans un projet Java sur BlueJ :



Lorsque les classes sont ajoutées, il va falloir créer un main pour lancer le programme :



Dans le code nous allons faire en sorte d'ajouter des auteurs, des livres et des adhérents puis d'ajouter le système d'emprunt pour que l'on puisse avoir des résultats dans l'affichage. Pour que cela marche j'ai dû faire modifications de la classe Livre pour faire marcher le constructeur, les getters et setters et l'affichage. De même pour les adhérents j'ai dû modifier le constructeur pour pouvoir le faire marcher.

Le code utilisé est le suivant :

```
public static void main(String[] args) {  
    Auteur camus = new Auteur("Camus", "Albert", new Date(), "Auteur philosophique");  
    Auteur flaubert = new Auteur("Flaubert", "Gustave", new Date(), "Romancier réaliste");  
  
    Livre l1 = new Livre("8573823224", "L'étranger", camus, 20);  
    Livre l2 = new Livre("7495948500", "Madame Bovary", flaubert, 19);  
    Livre l3 = new Livre("1234567890", "La Peste", camus, 18);  
  
    List<Livre> livresBibliotheque = new ArrayList<>();  
    livresBibliotheque.add(l1);  
    livresBibliotheque.add(l2);  
    livresBibliotheque.add(l3);  
  
    Adherent lionel = new Adherent(1, "Lionel", "Dupont");  
    Adherent alice = new Adherent(2, "Alice", "Martin");  
  
    List<Adherent> adherentsBibliotheque = new ArrayList<>();  
    adherentsBibliotheque.add(lionel);  
    adherentsBibliotheque.add(alice);  
  
    lionel.ajouterLivre(l2);  
    alice.ajouterLivre(l1);  
    alice.ajouterLivre(l3);  
  
    System.out.println("\n\n Liste des adhérents :");  
    for (Adherent a : adherentsBibliotheque) {  
        System.out.println("- " + a.getPrenom() + " " + a.getNom());  
    }  
  
    System.out.println("\n\n Liste des livres de la bibliothèque :");  
    for (Livre l : livresBibliotheque) {  
        l.Afficher();  
    }  
  
    System.out.println("\n\n Livres empruntés par chaque adhérent :");  
    for (Adherent a : adherentsBibliotheque) {  
        System.out.println(a.getPrenom() + " " + a.getNom() + " a emprunté :");  
        for (Livre l : livresBibliotheque) {  
            if (a.getLivre(l.getISBN()) != null) {  
                l.Afficher();  
            }  
        }  
        System.out.println();  
    }  
}
```

Résultat :

```

- Liste des adhérents :
- Lionel Dupont
- Alice Martin

- Liste des livres de la bibliothèque :
ISBN: 8573823224 | Titre: L'étranger | Auteur: Albert Camus | Prix: 20€
ISBN: 7495948500 | Titre: Madame Bovary | Auteur: Gustave Flaubert | Prix: 19€
ISBN: 1234567890 | Titre: La Peste | Auteur: Albert Camus | Prix: 18€

- Livres empruntés par chaque adhérent :
Lionel Dupont a emprunté :
ISBN: 7495948500 | Titre: Madame Bovary | Auteur: Gustave Flaubert | Prix: 19€

Alice Martin a emprunté :
ISBN: 8573823224 | Titre: L'étranger | Auteur: Albert Camus | Prix: 20€
ISBN: 1234567890 | Titre: La Peste | Auteur: Albert Camus | Prix: 18€

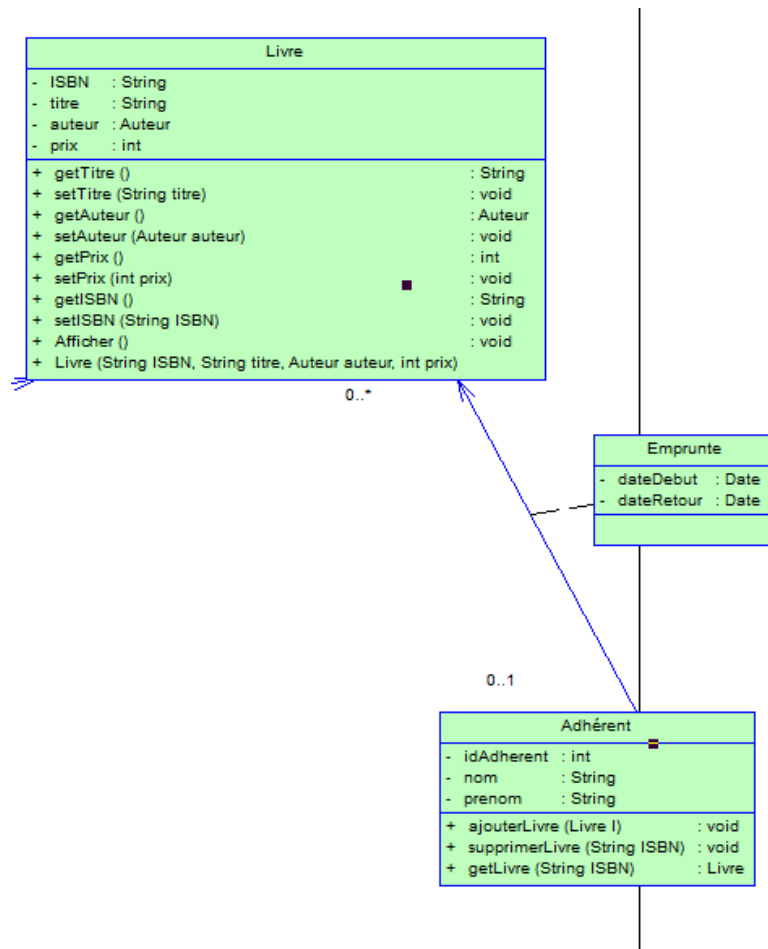
```

4) TD5

Nous voulons ajouter un système d'emprunt à notre diagramme, pour cela nous allons créer une classe-association entre Adhérent et Livre, pour faire cela nous allons créer une nouvelle classe nommé **Emprunte** puis nous allons double cliquer sur l'association Adhérent - Livre et ajouter Emprunte en tant que classe d'association :

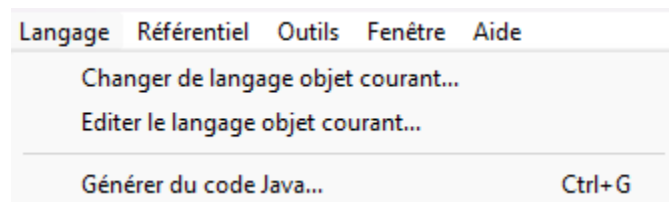


Le résultat devrait être le suivant :

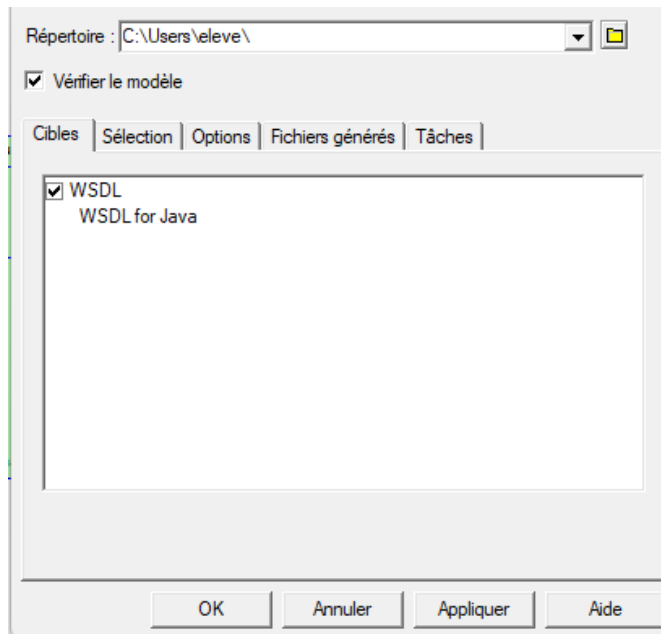


Et maintenant, voici comment générer le code Java des classes :

Il faut aller dans **Langage > Générer du code Java** :



Ensuite on ajoute un répertoire et on clique sur appliquer :



Puis on clique sur **OK**.

Cela va nous afficher les .java, et à partir de là on peut observer les codes de chaque classe :

```
C:\Users\veleve\Adherent.java  
C:\Users\veleve\Auteur.java  
C:\Users\veleve\Emprunte.java  
C:\Users\veleve\Livre.java
```

5) Conclusion

En conclusion, j'ai appris à créer un diagramme de classe à l'aide de PowerAMC, j'ai compris comment l'utiliser pour créer les classes, les associations et les classes-association. Cependant je n'ai pas réussi à faire le système de dictionnaire car l'option HashMap ne m'est pas proposée pour l'association entre Livre et Adhérent.

